Food Store en la feria

TP8 de la cátedra · Unidad 8 · Santiago Barretto · DB_Santiago_Barretto · 04/11/2026

Alcance y regla de autoridad

Se trabajó con foodstore_tp10_u8, copia de PostgreSQL Food Store usada en TP9, y con SQLite 3.49.1. Los cuatro productos son sintéticos; el 100001 también aparece en los pedidos MongoDB de Unidad 7. El móvil consulta y arma borradores; PostgreSQL decide precio, stock y ventas. SIMULADO_APROBADO no es una venta.

1.a · Qué datos viajan

A la tablet	No a la tablet
ID, nombre, precio en centavos enteros, stock visto, disponible, eliminado y fecha de copia.	Clientes, cuentas, contraseñas, correos, teléfonos, costos internos e historial completo de pedidos.

1.b · Esquema local

Tabla STRICT	Reglas principales
producto_cache	Centavos y stock no negativos; disponible/eliminado 0 o 1; lápida conservada.
pedido_borrador	UUID local; usuario sintético, forma de pago permitida y fecha.
item_borrador	FK a borrador y caché; cantidad > 0; precio visto en centavos.
cola_sync	UUID único; estados PENDIENTE, CONFLICTO, RECHAZADO o SIMULADO_APROBADO.
snapshot_servidor	Instantánea importada del CSV; contiene también productos dados de baja.

1.c · Transacciones, catálogo y errores

C1: un BEGIN/COMMIT guardó borrador, ítem y solicitud; otro BEGIN/ROLLBACK no dejó fila. El catálogo devolvió cuatro productos sin conexión al servidor. C7: cantidad cero produjo error CHECK y producto inexistente error FK; el conteo de ítems siguió en uno. PRAGMA foreign_keys estuvo activo y las cinco tablas se crearon como STRICT.

2. Sincronización y semáforo

2.a · Instantánea PostgreSQL → CSV → SQLite

La vista v_snapshot_movil en la copia exportó los productos 100001-100004. El CSV se importó en snapshot_servidor y actualizó producto_cache en una transacción. Las cuatro versiones de la instantánea están en evidencia/snapshots/; la última conserva la lápida 100003. Fecha de copia 2026-11-04T13:50:19.737Z.

2.b · Regla ejecutada

El SQL procesa solamente solicitudes PENDIENTE. Rojo → RECHAZADO si el ítem falta, está no disponible, eliminado o excede stock; si no hay rojo, amarillo → CONFLICTO por precio distinto; si tampoco hay amarillo, verde → SIMULADO_APROBADO. La decisión y el motivo se escriben sólo en cola_sync.

2.c · Casos ejecutados

Caso y comando/preparación	Resultado real y explicación
C2: UPDATE de precio en copia (100001: 53700 → 54700 centavos), nueva instantánea; luego UPDATE del precio visto e INSERT con otro UUID.	CONFLICTO; tras aceptar, SIMULADO_APROBADO. El UUID anterior conserva el conflicto.
C3: UPDATE stock=2 y precio+100 centavos para 100002; borrador de 5 unidades.	RECHAZADO por stock insuficiente. Rojo prevaleció aunque también cambió el precio.
C4: UPDATE activo=false, deleted_at=clock_timestamp() para 100003; nueva instantánea y recarga.	RECHAZADO. El producto no aparece como disponible, pero su fila queda con eliminado=1.
C5: INSERT de borrador de 100004 sin cambios en la instantánea.	SIMULADO_APROBADO; sólo una resolución local.
C6: INSERT de solicitud con UUID de C5 y segunda corrida del semáforo.	Error UNIQUE; una fila por UUID y cero cambios al repetir la simulación.

Control PostgreSQL: antes y después de cada semáforo coincidieron los 200.000 pedidos, la suma de stock y el hash de precio/stock/actividad/lápida de los cuatro productos. Los cambios entre C2-C4 fueron preparación explícita en la copia; la base original foodstore quedó sin cambios.

3. Integración de tres motores

3.a · Mapa de datos y autoridad

Motor	Qué guarda	Quién manda
PostgreSQL	Productos, precios, stock, clientes, pedidos y detalles.	Fuente de verdad; el TP usa una copia de prueba.
MongoDB U7	20 pedidos y 40 líneas embebidas; ocho pedidos contienen 100001.	Copia documental de lectura. No autoriza ventas.
SQLite U8	Caché, instantánea, borradores, ítems y cola.	Autoridad sólo sobre borradores locales; el semáforo es simulado.

3.b · Mantenimiento y restauración

Pregunta	Plan
¿Qué se respalda?	PostgreSQL: pg_dump custom diario, SHA-256 y restauración semanal de prueba como en U6. Borradores/cola de tablets: copia cifrada al cierre. MongoDB derivado: respaldo propio si se usa, o reconstrucción contrastada desde PostgreSQL.
¿Servidor restaurado desde copia vieja?	Detener envíos automáticos; conservar UUID locales, descargar una instantánea nueva y revalidar todo en el servidor. Ningún SIMULADO_APROBADO antiguo se convierte en venta.
¿Cuándo se revisan conflictos?	Responsable de operaciones: al recuperar conexión, cada 30 minutos durante feria activa y al cierre diario. Requiere aceptación humana o cancelación.

La idempotencia real iría en el servidor: tabla de solicitud_uuid única y transacción que revalide precio/stock, cree el pedido y descuente stock sólo una vez. No se implementó esa venta en la tablet.

3.c · Code review con OpenCode Plan

OpenCode inventarió scripts SQL, JS, PowerShell y Python de varias unidades. Se contrastaron cuatro observaciones con archivos y con el laboratorio; la respuesta original y la decisión detallada están en evidencia/ y docs/revision_opencode.md.

Unidad y hallazgo	Decisión y prueba
U8: conteo de pedidos + suma de stock no detectaban un cambio de precio o baja.	CORREGIDO: hash de los cuatro productos antes/después de cada semáforo; coincide en C1-C5.
U7: un fallo parcial de insertMany puede dejar colección y bloquear el reintento.	ACEPTADO como riesgo; el guardia impide sobrescritura. Se documentó inspección y recuperación manual; la colección válida tiene 20 documentos.
U6: la última línea de SELECT EXISTS podría ser un aviso; se sugirió riesgo de interpolación.	DESCARTADO para este ensayo: nombres constantes, salidas reales de una línea t. No se altera un TP previo sin fallo reproducido.
U8: el runner no reejecuta sobre SQLite existente.	ACEPTADO como protección: exige copia fresca y no pisa borradores/evidencia.

Una sospecha adicional de IA fue descartada: SIMULADO_APROBADO no se trata como venta; semaforo.sql únicamente modifica la cola SQLite. El conteo de pedidos y el estado del servidor se verifican antes y después de cada corrida.

4. Spec, bitácora y defensa

4.a · Spec previa en formato Kiro

EARS	Comportamiento y caso
REQ-01	WHEN se crea un borrador, guardar todo en transacción; invalidaciones sin filas parciales. C1/C7.
REQ-02	WHEN cambia precio sin rojo, marcar CONFLICTO; con precio aceptado y UUID nuevo, aprobación simulada. C2.
REQ-03	WHEN falta producto, hay baja o stock insuficiente, marcar RECHAZADO sin tocar el servidor. C3/C4.
REQ-04	WHEN todo coincide, marcar SIMULADO_APROBADO una vez; UUID repetido falla. C5/C6.

docs/kiro/requirements.md, design.md y tasks.md se escribieron antes de los scripts. El formato es compatible con la actividad; no se afirma haber usado la aplicación Kiro. OpenCode Plan propuso el esquema y el semáforo a partir de la spec.

4.b · Bitácora de IA y DUIA

Tres entradas reales en docs/bitacora_ia.md: (1) se aceptó la prioridad del semáforo y se corrigieron los campos usuario/forma_pago omitidos por la IA; (2) se corrigió la verificación de PostgreSQL con hash; (3) se documentó el riesgo de importación parcial MongoDB y se descartó el supuesto fallo concreto del script U6. docs/DUIA.md declara Codex y OpenCode, decisiones y pruebas.

4.c · Defensa

docs/ensayo_defensa.md prepara diez minutos: mapa, C2 en vivo y hallazgo corregido. La presentación en clase corresponde al estudiante y no se declara como realizada por este informe.

Cierre

SQLite permite consultar una copia local y guardar borradores con integridad. La prueba demostró conflicto, rechazo, lápida e idempotencia local sin crear ventas. PostgreSQL debe conservar la decisión final; MongoDB U7 sigue como lectura documental. El informe se acompaña de scripts, CSV, DB SQLite y evidencia en la rama codex/tp10-unidad8-sqlite.

Fuentes técnicas: SQLite STRICT, SQLite foreign_keys y PostgreSQL CREATE VIEW. Salidas ejecutadas: evidencia/resultado_laboratorio.txt.